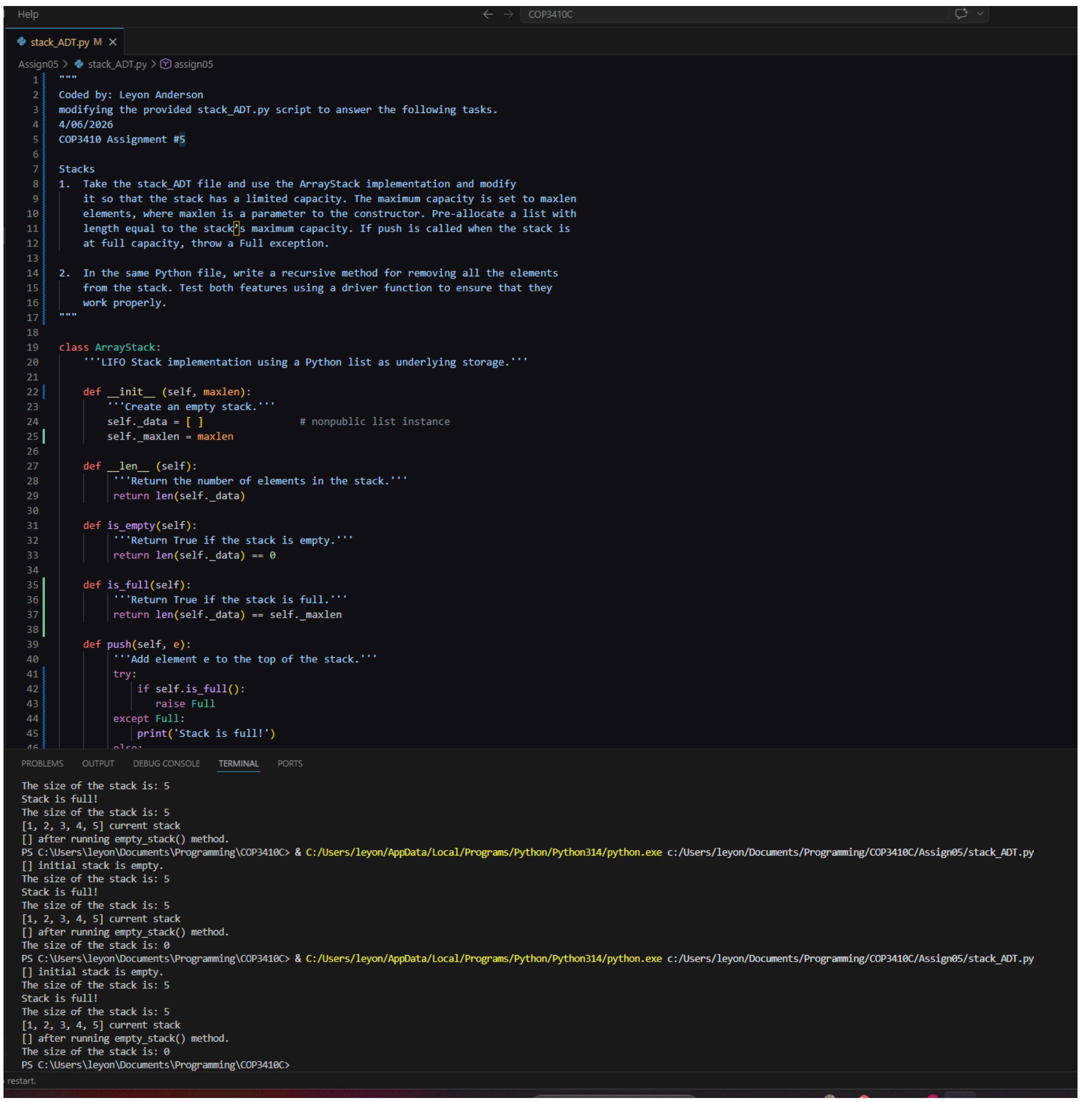
COP3410 Florida Atlantic University Assignment 5

100 points

Stacks

1. Take the stack_ADT **file and use the** ArrayStack **implementation and modify it so that the stack has a limited capacity. The maximum capacity is set to** maxlen **elements, where** maxlen **is a parameter to the constructor. Pre-allocate a list with length equal to the stack's maximum capacity. If push is called when the stack is at full capacity, throw a** Full **exception.**

2. In the same Python file, write a recursive method for removing all the elements from the stack. Test both features using a driver function to ensure that they work properly.

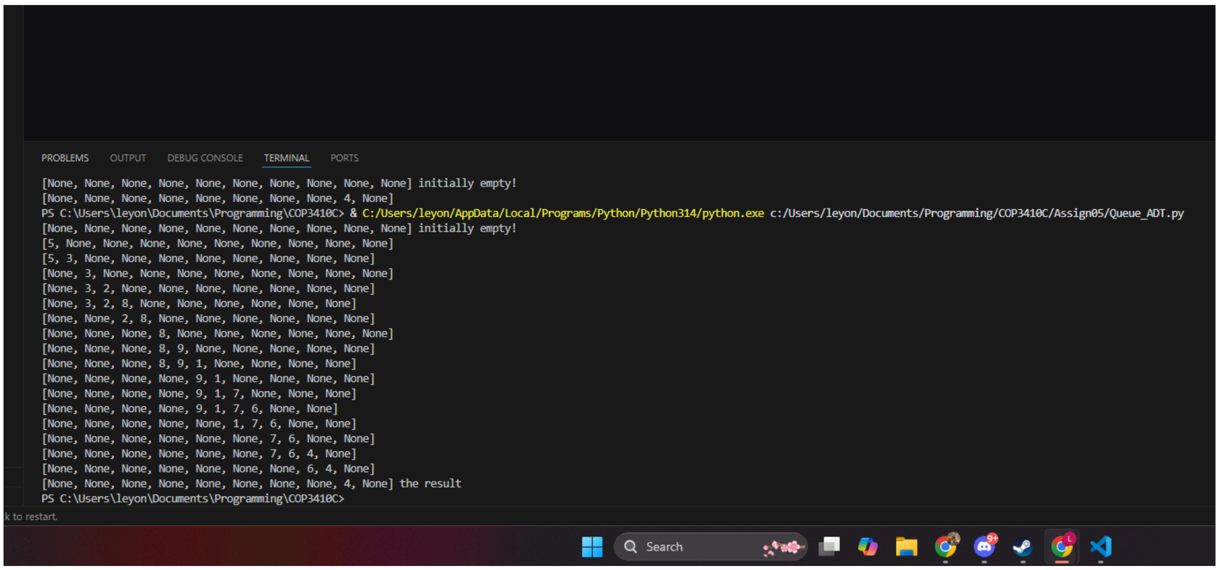


```python
"""
Coded by: Leyon Anderson
modifying the provided stack_ADT.py script to answer the following tasks.
4/06/2026
COP3410 Assignment #5

Stacks
1.  Take the stack_ADT file and use the ArrayStack implementation and modify
    it so that the stack has a limited capacity. The maximum capacity is set to maxlen
    elements, where maxlen is a parameter to the constructor. Pre-allocate a list with
    length equal to the stack's maximum capacity. If push is called when the stack is
    at full capacity, throw a Full exception.

2.  In the same Python file, write a recursive method for removing all the elements
    from the stack. Test both features using a driver function to ensure that they
    work properly.
"""

class ArrayStack:
    '''LIFO Stack implementation using a Python list as underlying storage.'''

    def __init__ (self, maxlen):
        '''Create an empty stack.'''
        self._data = [ ]                # nonpublic list instance
        self._maxlen = maxlen

    def __len__ (self):
         '''Return the number of elements in the stack.'''
         return len(self._data)

    def is_empty(self):
         '''Return True if the stack is empty.'''
         return len(self._data) == 0

    def is_full(self):
         '''Return True if the stack is full.'''
         return len(self._data) == self._maxlen

    def push(self, e):
         '''Add element e to the top of the stack.'''
         try:
             if self.is_full():
                raise Full
         except Full:
             print('Stack is full!')
```

```
The size of the stack is: 5
Stack is full!
The size of the stack is: 5
[1, 2, 3, 4, 5] current stack
[] after running empty_stack() method.
PS C:\Users\leyon\Documents\Programming\COP3410C> & C:/Users/leyon/AppData/Local/Programs/Python/Python314/python.exe c:/Users/leyon/Documents/Programming/COP3410C/Assign05/stack_ADT.py
[] initial stack is empty.
The size of the stack is: 5
Stack is full!
The size of the stack is: 5
[1, 2, 3, 4, 5] current stack
[] after running empty_stack() method.
The size of the stack is: 0
PS C:\Users\leyon\Documents\Programming\COP3410C> & C:/Users/leyon/AppData/Local/Programs/Python/Python314/python.exe c:/Users/leyon/Documents/Programming/COP3410C/Assign05/stack_ADT.py
[] initial stack is empty.
The size of the stack is: 5
Stack is full!
The size of the stack is: 5
[1, 2, 3, 4, 5] current stack
[] after running empty_stack() method.
The size of the stack is: 0
PS C:\Users\leyon\Documents\Programming\COP3410C>
```

Queues

3. What values are returned during the following sequence of queue operations, if executed on an initially empty queue? enqueue(5), enqueue(3), dequeue(), enqueue(2), enqueue(8), dequeue(), dequeue(), enqueue(9), enqueue(1), dequeue(), enqueue(7), enqueue(6), dequeue(), dequeue(), enqueue(4), dequeue(), dequeue(). Use the Queue.py **file to show the queue contents after every operation.**



4. Suppose an initially empty queue Q has executed a total of 32 enqueue operations, 10 first operations, and 15 dequeue operations, 5 of which raised Empty **errors that were caught and ignored. What is the current size of Q? Add your answer to the end of the python file.**

Queue_ADT.py M

Assign05 > Queue_ADT.py > ...

```python
    print(A)
    A.dequeue()
    print(A)
    A.enqueue(9)
    print(A)
    A.enqueue(1)
    print(A)
    A.dequeue()
    print(A)
    A.enqueue(7)
    print(A)
    A.enqueue(6)
    print(A)
    A.dequeue()
    print(A)
    A.dequeue()
    print(A)
    A.enqueue(4)
    print(A)
    A.dequeue()
    print(A)
    A.dequeue()
    print(f'{A} the result')

    """
    4.  Suppose an initially empty queue Q has executed a total of 32 enqueue operations,
        10 first operations, and 15 dequeue operations, 5 of which raised Empty errors
        that were caught and ignored. What is the current size of Q? Add your answer to
        the end of the python file.

        Ths size of Q should be 32 - 10 = 22, assuming 5 failed dequeue operations. First
        operations would not impact size.
    """
```